

Project assignment

Software Engineering Course

Master Degree in Computer Engineering

University of Salerno

2023/2024



Problem statement

- ✦ Develop a simple "task automation" application, similar in spirit to IFTTT (see <http://ifttt.com>), but simpler
- ✦ Starting point: the User Epics given at the end of this presentation



The task

- ✦ Groups of 4 students
- ✦ The groups define their user stories
 - ✦ Based on the provided User Epics
 - ✦ Teachers may add or change the user stories during the project
- ✦ The project will be performed using the Scrum process
- ✦ **TIME BOXING:** the program does not need to be complete at the end of the project!



Tools

- ✦ The groups must set up a Git repository for the project on GitHub
- ✦ The repository must contain
 - ✦ Source code (obviously), including unit tests (based on JUnit)
 - ✦ A (short) document describing the software architecture
 - ✦ The product backlog
 - ✦ The sprint backlogs for each sprint
 - ✦ (At the end) A presentation of the project describing both the product and the process



Tools

- ✦ The team will use Trello (<http://trello.com>) as a platform for tracking the project activities (Sprint Task Board) and their completion
 - For each sprint, the tasks assigned to each member of the team and the time spent on them must be tracked
- ✦ Links:
 - <https://blog.trello.com/how-to-scrum-and-trello-for-teams-at-work>



The deadlines

♦ First delivery: Sun. November 19

- ♦ Pre-game: Initial product backlog + 1st sprint planning and backlog + Description of software architecture (5 min. presentation per group)

♦ Second delivery: Sun. November 26

- ♦ 1st Sprint Release + Sprint backlog + Updated prod. backlog + Project Burndown chart + Sprint review report + Sprint retrospective report

♦ Third delivery: Sun. December 3

- ♦ 2nd Sprint Release + Sprint backlog + Updated prod. backlog + Project Burndown chart + Sprint review report + Sprint retrospective report

♦ Fourth delivery: Sun. December 10

- ♦ 3rd Sprint Release + Sprint backlog + Updated prod. backlog + Project Burndown chart + Sprint review report + Sprint retrospective report + Final presentation (8 min. per group)



Classroom activity

- ✦ Two of the three weekly lectures will be used for short demonstrations of the software released on the previous week (during these demos you may be asked to show also the product/sprint backlogs, review/retrospective reports, burndown chart)
- ✦ A final demonstration/presentation will be given by each group in the week between Dec 11 and Dec 15



Artifacts: backlogs

- ✦ The product backlog and sprint backlogs should be kept or copied in a Word or Excel document for ease of inspection by the teachers
 - You may use a Google Docs/Sheets shared document, and have in your GitHub repo a file/document containing the links
- ✦ Don't delete completed items from the backlog; just move them to a different part of the doc



Artifacts: burndown chart

- ✦ You should make the burndown chart (or the "extended" burndown chart) for tracking the whole project (not the sprint burndown chart)



Artifacts: review report

- ✦ Keep it short (max 1 page)
- ✦ Essential information:
 - Story Points planned vs completed, Project Velocity
 - Short discussion of issues discovered implementing/examining the release (e.g. bugs, technical problems, technical debts)
 - Short discussion of issues with the Product Backlog (e.g. need to add/refine/modify some user stories)



Artifacts: retrospective report

- ✦ Keep it short (max 1 page)
- ✦ Follow the "starfish diagram"
 - Start / More of / Keep doing / Less of / Stop
- ✦ Keep it about the process (not the product)



Time organization

- ✦ Each member of the group is expected to work on the project for **10 hours** per week
- ✦ Better to have these hours divided between two or three days (agreed among the members of the team)



Time organization

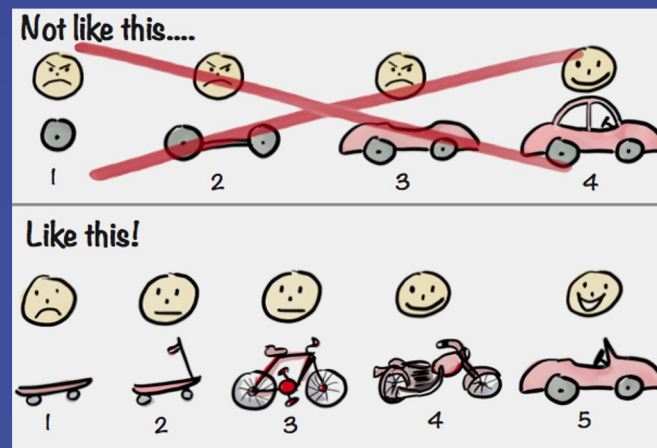
- ✦ **Pre-Game: the members of the team will work mostly together**
 - Initial product backlog: the members of the team will fill the role of the Product Owner
 - ▶ Try to keep a shared vision of the product
 - Tools setup: GitHub repo, Trello, coding conventions, development platforms, external libraries (if needed)
 - Definition of Done
 - Design of the architecture of the program
 - First sprint planning (initial estimate of velocity, selected user stories)
 - If needed, get acquainted with the architecture and technology chosen (e.g. develop a simple "Hello world"-like program to check that everything is in place for compiling and running code following your architecture/technology/libraries etc.)



Time organization

✦ At the beginning of each sprint

- If necessary, Product Backlog grooming: less than 1 hour, together
 - ▶ Try to address the issues discovered during the previous sprint review
- Sprint planning: 1.5 hours, together
 - ▶ Choose user stories for current sprint
 - ▶ Define and assign tasks (requires some design)
 - ▶ When dividing a user story into tasks, ensure that there are "intermediate checkpoints" where, even if the story is not completed, the code compiles and passes its tests



Time organization

- ✦ At the beginning of each workday within the sprint
 - Daily meeting: 5 minutes, together
 - ▶ What you did last day, what you are going to do today, problems/obstacles you have encountered
 - Refine and discuss design decisions: 5-30 minutes, subsets of the team
 - ▶ Discuss decisions with other members whose work is going to be impacted
 - ▶ Agree on the interfaces between the different parts



Time organization

- ✦ At the end of each workday within the sprint
 - Take the time needed to integrate the code that is ready with the rest of the release (you can do this more often than daily)



Time organization

✦ At the end of each each sprint

- Code review: 30 minutes, individually
 - ▶ Examine the code produced by other team members, to spot problems in design or coding (see the checklists later in this presentation)
 - ▶ Better to agree on who reviews what, so as to avoid duplicate work
 - ▶ Not necessary to have a 100% coverage, but try to have a fair sampling (for instance, ensure that work by a team member is reviewed by a different colleague at each sprint)
 - ▶ Problems found can be discussed in the Sprint Review



Time organization

- ✦ At the end of each each sprint
 - Sprint Review: 40 minutes, together
 - Sprint Retrospective: 20 minutes, together



Evaluation criteria

- ✦ The rating of each student will be based on the individual tasks performed by the student (as documented in each iteration) and on the overall evaluation of the project
 - Individual tasks: 50%
 - Overall project evaluation: 50%
- ✦ Each member of the group will be asked at the end to provide (confidentially) an estimate of the relative effort contributed by each other member
 - No "free riders"!



Evaluation criteria

- ✦ Appearance and usability: 10%
- ✦ Quality of the design: 30%
- ✦ Quality of the coding: 15%
- ✦ Quality of the tests: 15%
- ✦ Compliance with the Scrum process: 30%
- ✦ Effort: a multiplicative coefficient that scales all of the above
 - You are expected to work **10 hours/week**, and divide the tasks among the team members!



Design checklist

- ✦ Are the packages/classes designed to provide a set of highly related services (**high cohesion**)?
- ✦ Are unnecessary dependencies avoided (**low coupling**)?
- ✦ Are abstraction mechanisms used to improve reusability and/or break dependency chains?
- ✦ Are **design patterns** used when appropriate?
- ✦ Are **duplications/repetitions** in the design avoided?
(Don't Repeat Yourself principle)



Coding checklist

- ✦ Are naming conventions followed consistently? Are names chosen so as to be easily understandable?
- ✦ Is code formatting (e.g. indentation) consistent and readable?
- ✦ Are "hard coded" constants/"magic" numbers in the code avoided?
- ✦ Is the code adequately commented? (Comments should describe the interfaces, and why you are doing things a certain way – not be redundant)
- ✦ Are duplications/repetitions in the code avoided? (Don't Repeat Yourself principle)
- ✦ Is the code structure readable? (e.g. split too complex operations into several methods)



Testing checklist

- ✦ Are the unit tests automated?
- ✦ Do the tests cover the public part of each class? (at least for business-logic methods / classes)
- ✦ Do the test cases cover adequately the input space (e.g. including limit values, special conditions etc.)



User Epics



WARNING!

- ✦ The following User Epics are in order of priority (most important first)
- ✦ You are not expected to complete ALL the User Epics within the project duration
 - You must complete as much as you can WITHOUT compromising on the quality



Basic operation of the program

The desired application is similar in spirit to IFTTT (<http://ifttt.com>), a cloud-based web service that helps in the automation of simple tasks; however, an important simplification is that the desired application works as a stand-alone, single-user desktop program.

The user can create a set of rules, where each rule has a trigger and an action. The application periodically (i.e. every few seconds) checks the rules to see if the trigger is verified, and if so, it executes the corresponding action (the rule is "fired"). The trigger is the specification of a particular time of the day (e.g. 11:37), which becomes true (firing the rule) when the current time is at or after the specified one. The action can be either the playing of a specified audio file, or the display of a specified message in a dialog box, that must be explicitly closed by the user.



Editing and permanent storage of the rules

In any moment, the user can add more rules, and can delete existing rules. Also, each rule can be made "inactive" (in the sense that it is not checked by the program, but it is still present) or reactivated.

The set of rules must be saved on a file, and automatically reloaded when the application is restarted.

The user can specify if a rule must be fired only once (i.e., it is not fired again even if the trigger becomes true again), or if it can be fired again after a "sleeping" period (specified in days, hours and minutes).



More actions and more triggers

The possible actions include: writing a specified string at the end of a specified text file; copying or moving a specified file from a specified source directory to a specified destination directory; deleting a specified file from a specified directory; executing a specified external program, with specified command-line arguments.

The possible triggers include: the current day-of-week is a specified day (e.g. "Monday"); the current day-of-month is a specified day (e.g. 22); the current date is a specified date (e.g. 18/12/2023).

The possible triggers also include: a file with a specified name exists in a specified directory; the size of a specified file is larger than a specified value.

The possible triggers also include the execution of a specified external program, with specified command-line arguments; the trigger is true if the exit status of the external program is equal to a specified value.

Note: in the text above, "specified" means "specified by the user"



Combined actions and triggers

The possible action for a rule can be a sequence of actions; the sequence may contain elementary actions, but also other sequences.

The possible trigger of a rule can be: the logical NOT of a trigger; the logical AND of two triggers; the logical OR of two triggers. The arguments of these logical operations can be elementary triggers, but also combined triggers. For instance, it is possible to have a trigger that is the AND between a trigger and the result of the OR between two other triggers: time-of-day is 11:37 AND day-of-week is Tuesday OR day-of-week is Friday.



Counters

→ set counters

The user can define counters, which are named variables having integer values; the user can also view and change the current value of a counter.

The possible actions include: setting a specified counter to a specified value; adding a specified number to the value of a specified counter; adding the value of a specified counter to another specified counter.

The possible triggers include: the value of a specified counter is equal to/less than/greater than a specified integer; the value of a specified counter is equal to/less than/greater than the value of another specified counter.

In all the actions involving a string (e.g. writing a string to a file or showing a message in a dialog box), it must be possible to use a form of "variable substitution" (e.g. like the use of the "\$" character in the Bash shell) to insert the current value of a counter in the string.

